

A Lagrangian Heuristic for Set-Covering Problems

J. E. Beasley

The Management School, Imperial College, London SW7 2AZ, England

In this article we present a heuristic algorithm for non-unicost set-covering problems based upon Lagrangian relaxation and subgradient optimization. Computational results, based upon problems involving up to 1000 rows and 10000 columns, indicate that this heuristic produces better-quality results than a number of other heuristics.

1. INTRODUCTION

The set-covering problem (SCP) is the problem of covering the rows of a m row, n column, zero-one matrix (a_{ij}) by a subset of the columns at minimum cost. Defining $x_j = 1$ if column j (cost $c_j > 0$) is in the solution ($x_j = 0$ otherwise) the SCP is:

$$\text{minimize} \quad \sum_{j=1}^n c_j x_j, \quad (1)$$

$$\text{subject to} \quad \sum_{j=1}^n a_{ij} x_j \geq 1, \quad i = 1, \dots, m, \quad (2)$$

$$x_j \in (0, 1), \quad j = 1, \dots, n. \quad (3)$$

Equation (2) ensures that each row is covered by at least one column and Eq. (3) is the integrality constraint.

It is well known that the SCP is NP-complete (e.g., see [14]) and a number of optimal algorithms, typically based upon tree-search procedures, have been presented in the literature (e.g., see [3, 4, 6, 8, 15, 18]). These algorithms have solved unicast ($c_j = 1, j = 1, \dots, n$) problems involving up to 50 rows and 500 columns and non-unicost problems involving up to 400 rows and 4000 columns (albeit at considerable computational cost, particularly for non-unicost problems).

It is clear that, given the current situation with respect to optimal algorithms, there is a place for a computationally effective heuristic algorithm capable of producing good-quality (near-optimal) solutions.

A number of heuristic algorithms for the SCP have been given in the literature. Many of these algorithms can, in general terms, be regarded as variations on the following general procedure (PSCP) for the SCP.

- (1) Let S be the set of columns chosen to be in the solution to the SCP (initially $S = \emptyset$).
- (2) Add to S the column j which minimizes some function $f(j, c_j, S)$ and repeat until S constitutes a solution to the SCP.
- (3) Improve S by (a) removing columns which are redundant; and (b) interchanging columns in S with columns not in S .

Note here that step (2) above is a greedy procedure.

Roth [16] presented an algorithm based on PSCP but choosing j randomly at step (2). Few computational results were given.

Chvatal [7] presented a worse-case analysis of the solution produced by step (2) of PSCP when $f(j, c_j, S) = c_j / (\text{number of rows covered by column } j \text{ that are not covered by } S)$. Ho [11] later extended Chvatal's analysis. Baker and Fisher [2] reported using such solutions as part of an algorithm for air crew scheduling problems (see also Baker [1]).

Hochbaum [12, 13] presented a worse-case analysis of the solution produced by a heuristic based upon solving the linear programming relaxation of the SCP.

Balas and Ho [3] presented a number of heuristic algorithms in the context of a tree-search procedure for the SCP. One of these heuristics (which we shall denote by BH) corresponds to PSCP where step (3b) is excluded and five different functions $f(j, c_j, S)$ are used.

Vasko and Wilson [17] presented a heuristic (SCHEURI) corresponding to PSCP where seven different functions $f(j, c_j, S)$ are used. They also presented a heuristic (SCFUNC1TO7) involving the same seven functions but where the function to be used each time a column j is to be chosen in step (2) of PSCP is randomly generated (see also Vasko and Wilson [18] and Vasko and Wilson [19] with respect to unicast problems).

In a recent article Vasko and Wolf [20] presented a number of heuristics based upon adaptations of SCHEURI and SCFUNC1TO7.

In this article we present a heuristic for non-unicost SCP's based upon Lagrangian relaxation and subgradient optimization. In the next section we outline this algorithm.

2. ALGORITHM OUTLINE

In a previous article [4] we presented a tree-search procedure for the SCP which incorporated a lower bound based upon Lagrangian relaxation and subgradient optimization. In this article we illustrate how (a) generating a lower bound for the SCP via Lagrangian relaxation; and (b) attempting to maximize that lower bound via subgradient optimization; produces a sequence of lower-bound solutions that can be modified, in a computationally effective way, to produce a sequence of feasible solutions to the SCP. In addition we can, through use of the lower bound and the value of the best feasible solution found, identify columns that can be removed from the problem.

Readers unfamiliar with Lagrangian relaxation and subgradient optimization are referred to Fisher [9, 10]. We shall develop the Lagrangian heuristic algorithm using the same notation as in [4].

Without loss of generality assume that the columns have been ordered in ascending cost order, with columns of equal cost being ordered in descending order of the number of rows that they cover. In essence this means that for any row i the column $\min[j \mid a_{ij} = 1, j = 1, \dots, n]$ is the "best" column to use in covering row i .

Letting t_i ($\geq 0, i = 1, \dots, m$) be the Lagrange multipliers associated with Eq. (2) we have the Lagrangian lower-bound program (LLBP) given by

$$\text{minimize} \quad \sum_{j=1}^n \left(c_j - \sum_{i=1}^m t_i a_{ij} \right) x_j + \sum_{i=1}^m t_i, \quad (4)$$

$$\text{subject to} \quad x_j \in (0,1), \quad j = 1, \dots, n. \quad (5)$$

Letting C_j represent the coefficient of x_j in Eq. (4) and (X_j) the solution values of the (x_j) we have that $X_j = 1$ if $C_j \leq 0$ ($X_j = 0$ otherwise) with a valid lower bound (Z_{LB}) on the optimal solution to the original SCP given by

$$Z_{LB} = \sum_{j=1}^n C_j X_j + \sum_{i=1}^m t_i. \quad (6)$$

We use subgradient optimization, in the manner presented below, in an attempt to maximize the lower bound obtained from LLBP.

Let $Z_{\max}$ represent the maximum lower bound found, Z_{UB} the best feasible solution found and P_k the lower bound when column k ($k = 1, \dots, n$) is forced to be in the solution.

(1) Initialize $Z_{\max} = -\infty$, $Z_{UB} = \infty$, $P_k = c_k$ ($k = 1, \dots, n$) and $t_i = \min[c_j \mid a_{ij} = 1, j = 1, \dots, n]$ ($i = 1, \dots, m$).

(2) Solve LLBP with the current set of multipliers (t_i) and, as before, let the solution be Z_{LB} , (X_j) . Update $Z_{\max}$ by $Z_{\max} = \max(Z_{\max}, Z_{LB})$.

(3) Construct a feasible solution S to the original SCP in the following way:

- (a) set $S = [j \mid X_j = 1, j = 1, \dots, n]$
- (b) for each row i which is uncovered (i.e., $\sum_{j=1}^n a_{ij} X_j = 0$) add the column corresponding to $\min[j \mid a_{ij} = 1, c_j < \infty, j = 1, \dots, n]$ to S
- (c) consider each column $j \in S$ in descending index (j) order and if $S - [j]$ is a feasible solution to the SCP set $S = S - [j]$
- (d) update Z_{UB} by $Z_{UB} = \min(Z_{UB}, \sum_{j \in S} c_j)$.

Step (b) above can be accomplished in a computationally efficient manner by noting that, neglecting columns deleted from the problem, we always choose the same column to cover an uncovered row.

(4) Stop if $Z_{\max} = Z_{UB}$ (Z_{UB} is the optimal solution).

(5) Considering the original SCP [Eqs. (1)–(3) above] and the corresponding LLBP [Eqs. (4) and (5) above] it is clear that imposing the additional constraint that a particular column k must be in the optimal solution ($x_k = 1$) results in a corresponding lower bound of $Z_{LB} + C_k$ if $X_k = 0$ [Z_{LB} otherwise ($X_k = 1$)]. Hence we can update P_k using

$$P_k = \max(P_k, Z_{LB} + C_k), \quad \text{if } X_k = 0, \quad k = 1, \dots, n, \quad (7)$$

$$P_k = \max(P_k, Z_{LB}), \quad \text{if } X_k = 1, \quad k = 1, \dots, n. \quad (8)$$

We can therefore remove columns from the problem by setting

$$c_k = \infty, \quad \text{if } P_k > Z_{UB}, \quad k = 1, \dots, n, \quad (9)$$

since plainly a column k with P_k (the lower bound corresponding to an optimal

solution containing column k) greater than Z_{UB} cannot be in an improved feasible solution.

(6) Calculate the subgradients (G_i) using

$$G_i = 1 - \sum_{j=1}^n a_{ij}X_j, \quad i = 1, \dots, m. \quad (10)$$

We found it computationally useful to adjust the subgradients, if they were not going to effectively contribute to the update of the multipliers, in the following manner:

$$G_i = 0, \quad \text{if } t_i = 0 \text{ and } G_i < 0, \quad i = 1, \dots, m. \quad (11)$$

(7) Stop if $\sum_{i=1}^m (G_i)^2 = 0$ [since in this case we cannot define a suitable step size [Eq. (12) below]].

(8) Define a step size T by

$$T = f(1.05Z_{UB} - Z_{LB}) / \left(\sum_{i=1}^m (G_i)^2 \right) \quad (12)$$

where $f = 2$ initially. If $Z_{\max}$ has not increased in the last 30 iterations of the subgradient procedure with the current value of f then f is halved. This approach to deciding the value of f is modeled on that of Fisher [9, 10]. Equation (12) is the standard expression for step size in subgradient optimization except for the factor 1.05 which (computationally) we have found useful in ensuring that T does not become too small as the gap between Z_{UB} and Z_{LB} closes.

(9) Stop if $f \leq 0.005$ (an arbitrarily decided stopping criteria).

(10) Update the Lagrange multipliers using

$$t_i = \max(0, t_i + TG_i), \quad i = 1, \dots, m \quad (13)$$

(the standard expression for updating nonnegative Lagrange multipliers) and go to step (2) to resolve the problem.

At the end of the above Lagrangian heuristic Z_{UB} is the value of the best feasible solution found and $Z_{\max}$ the value of the best lower bound (on the optimal solution to the original SCP) found.

The gap between Z_{UB} and $Z_{\max}$ (e.g., $(Z_{UB} - Z_{\max})/Z_{\max}$) gives an *indication* of the quality of the solution (Z_{UB}) obtained. If it is felt that this gap is too large then one obvious strategy in an attempt to improve Z_{UB} would be to resort to an interchange procedure (i.e., examine interchanges between columns in the best solution found and other columns). This is computationally more attractive than it otherwise would be since the Lagrangian heuristic eliminates some columns completely [step (5) above] and so they can be ignored without affecting the optimal solution.

Note here that we implicitly use the ordering we have imposed upon the columns when constructing a feasible solution to the SCP [steps (3b) and (3c) above] at each subgradient iteration. If we were to apply the heuristic algorithm

given above to unicast problems we would not expect this ordering to be sufficiently discriminating between columns to give good-quality results. Limited computational experience has confirmed this expectation and so we would not recommend applying the Lagrangian heuristic presented in this article to unicast problems.

2.1. Extensions

It is natural to ask whether the Lagrangian heuristic presented in this article can be extended to deal with the set-covering problem with side conditions (additional constraints). Although we have no computational experience with such problems it is clear that the basic approach of the Lagrangian heuristic, namely

- (a) choose a Lagrangian relaxation of the problem which (computationally) is easy to solve
- (b) use subgradient optimization in an attempt to maximize the lower bound obtained from the relaxation
- (c) at each subgradient iteration adjust (if possible) the current lower-bound solution into a feasible solution of the original problem

is applicable to the set-covering problem with side conditions.

The principal difficulty with this approach is (c) above, devising a procedure for efficiently adjusting lower-bound solutions into good-quality feasible solutions for the original problem. However, such an approach has been found to be computationally successful in a number of different problem areas (e.g., see [5, 9, 10]).

3. EXAMPLE

In order to illustrate the Lagrangian heuristic we present below an example of the application of the heuristic to a small problem. For reasons of space we neglect steps in the algorithm which lead to no positive result. Problem: $m = 3$; $n = 4$; $[c_j] = [2, 3, 4, 5]$ (ascending cost order);

$$a_{ij} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix}$$

optimal solution of cost 5 (columns 1 and 2).

Algorithm

Step	Result
1	$Z_{\max} = -\infty$; $Z_{\text{UB}} = \infty$; $[P_k] = [2, 3, 4, 5]$; $[t_i] = [2, 2, 3]$
2	$[C_j] = [-2, 0, -1, 0]$; $Z_{\text{LB}} = 4$; $[X_j] = [1, 1, 1, 1]$; $Z_{\max} = 4$
3(a)	$S = [1, 2, 3, 4]$

- 3(c) remove columns 4 and 3 from S to give $S = [1, 2]$
 3(d) $Z_{UB} = 5$
 5 $[P_k] = [4, 4, 4, 5]$; no columns removed from the problem
 6 $[G_i] = [-1, -1, -2]$
 8 $T = 2(1.05(5) - 4)/((-1)^2 + (-1)^2 + (-2)^2) = 0.417$
 10 $[t_i] = [1.583, 1.583, 2.166]$
 2 $[C_j] = [-1.166, 0.834, 0.251, 1.251]$; $Z_{LB} = 4.166$; $[X_j] = [1, 0, 0, 0]$;
 $Z_{max} = 4.166$
 3(a) $S = [1]$
 3(b) for row 3 add column 2 to S to give $S = [1, 2]$
 3(d) $Z_{UB} = 5$
 5 $[P_k] = [4.166, 5, 4.417, 5.417]$; column 4 removed from the problem
 so $[c_j] = [2, 3, 4, \infty]$
 6 $[G_i] = [0, 0, 1]$
 8 $T = 2(1.05(5) - 4.166)/(0^2 + 0^2 + 1^2) = 2.168$
 10 $[t_i] = [1.583, 1.583, 4.334]$
 2 $[C_j] = [-1.166, -1.334, -1.917, \infty]$; $Z_{LB} = 3.083$; $[X_j] = [1, 1, 1, 0]$;
 $Z_{max} = 4.166$
 3(a) $S = [1, 2, 3]$
 3(c) remove column 3 from S to give $S = [1, 2]$
 3(d) $Z_{UB} = 5$
 5 $P_k = [4.166, 5, 4.417, \infty]$; no further columns removed from the prob-
 lem
 6 $G_i = [-1, 0, -1]$
 8 $T = 2(1.05(5) - 3.083)/((-1)^2 + 0^2 + (-1)^2) = 2.167$
 10 $[t_i] = [0, 1.583, 2.167]$
 2 $[C_j] = [0.417, 0.833, 1.833, \infty]$; $Z_{LB} = 3.75$; $[X_j] = [0, 0, 0, 0]$; $Z_{max} =$
 4.166
 3(a) S is empty
 3(b) $S = [1, 2]$
 3(d) $Z_{UB} = 5$
 5 $P_k = [4.167, 5, 5.583, \infty]$; column 3 removed from the problem so $[c_j] =$
 $[2, 3, \infty, \infty]$
 6 $G_i = [1, 1, 1]$
 8 $T = 2(1.05(5) - 3.75)/(1^2 + 1^2 + 1^2) = 1$
 10 $[t_i] = [1, 2.583, 3.167]$
 2 $[C_j] = [-1.583, -0.167, \infty, \infty]$; $Z_{LB} = 5$; $[X_j] = [1, 1, 0, 0]$; $Z_{max} = 5$
 3(a) $S = [1, 2]$
 3(d) $Z_{UB} = 5$
 4 $Z_{max} = Z_{UB}$ so stop; Z_{UB} is the optimal solution.

4. COMPUTATIONAL RESULTS

The Lagrangian heuristic presented in this article was programmed in FORTRAN and run on a Cray-1S using the CFT compiler (with maximum optimization) for a number of test problems.

Table 1 presents a summary of these test problems with problem sets 4, 5, and 6 being taken from Balas and Ho [3], problem sets A, B, C, and D being

Table 1. Test problem details.

Problem set	Number of rows (m)	Number of columns (n)	Density (%)	Number of problems in problem set
4	200	1000	2	10
5		2000		
6		1000	5	5
A	300	3000	2	
B			5	
C	400	4000	2	
D			5	
E	500	5000	10	
F			20	
G	1000	10000	2	
H			5	
VW.1-4	100	400	5/10/15/20	4
VW.5-8		700		
VW.9-12		1000		
VW.13-16	150	1000		
VW.17-20	200	400		
VW.21-24		700		
VW.25-28		1000		
VW.29-32	100	2000		

Note: The density of a SCP is the percentage of ones in the (a_{ij}) matrix.

taken from Beasley [4] and problem set VW being taken from Vasko and Wilson [18] (see also Vasko and Wolf [20]). All of the test problems (except problem set VW) have been produced using the scheme of Balas and Ho [3], namely, column costs (c_j) are integers randomly generated from $[1, 100]$; every column covers at least one row; every row is covered by at least two columns.

In order to compare the heuristic algorithm presented in this article with other heuristics we also coded in FORTRAN the BH heuristic of Balas and Ho [3] involving using five different functions $f(j, c_j, S)$ and choosing the best solution found; the SCHEURI heuristic of Vasko and Wilson [17] involving using seven different functions $f(j, c_j, S)$ and choosing the best solution found; and the SCFUNC1TO7 heuristic of Vasko and Wilson [17] involving a random choice from seven different functions $f(j, c_j, S)$ (we executed SCFUNC1TO7 ten times and choose the best solution found).

Note here that we modified the Vasko and Wilson [17] heuristics (SCHEURI and SCFUNC1TO7) slightly in that the final step in both of these heuristics involves a 1-opt neighborhood search on a feasible solution S to the original SCP. Given the usual definition of a 1-opt neighborhood search this means that the number of columns in the solution ($|S|$) cannot change. We therefore modified this step so that, as well as examining columns $i \in S, j \notin S$ for possible exchange (the usual 1-opt neighborhood search) we also remove from S any columns that become redundant as a result of any such exchanges that are made.

The results for problem sets 4-6 are shown in Table 2, the results for problem sets A-D are shown in Table 3, the results for problem sets E-H are shown in Table 4 and the results for problem set VW in Table 5. As in [3] and [4] we

Table 2. Computational results—problem sets 4, 5, 6.

Problem number	Optimal solution value	Balas and Ho (BH)		Vasko and Wilson				Lagrangian heuristic				
				SCHEURI		SCFUNCITO7				Maximum		
		Solution value	Time	Solution value	Time	Solution value	Time	Solution value (Z_{UB})	Time	lower bound (Z_{max})	Number of subgradient iterations	Number of columns
4.1	429	449	0.07	433	1.59	446	2.19	429	2.16	428.98	581	79
4.2	512	545	0.07	527	1.54	566	2.12	512	2.18	512	469	60
4.3	516	551	0.06	534	1.48	570	2.10	516	1.34	516	331	64
4.4	494	532	0.07	504	1.60	558	2.13	495	2.39	493.93	640	108
4.5	512	528	0.06	518	1.49	543	2.05	512	1.38	512	332	60
4.6	560	609	0.07	568	1.56	581	2.17	561	3.11	557.17	761	143
4.7	430	450	0.06	442	1.45	446	2.01	430	1.81	429.97	567	70
4.8	492	512	0.06	508	1.43	508	2.01	493	2.25	488.61	643	131
4.9	641	711	0.07	664	1.66	667	2.25	641	4.04	638.29	828	136
4.10	514	590	0.07	519	1.53	539	2.14	514	2.34	513.42	591	87
5.1	253	268	0.14	268	2.81	265	3.84	255	3.42	251.16	691	180
5.2	302	326	0.13	317	2.94	340	4.03	304	4.11	299.62	745	206
5.3	226	246	0.14	230	2.67	268	3.69	226	1.98	225.98	467	64
5.4	242	256	0.14	246	2.84	270	3.92	242	2.26	240.48	526	132
5.5	211	226	0.13	212	2.80	229	3.78	211	1.42	210.96	424	72
5.6	213	236	0.14	216	2.82	249	3.65	213	2.23	212.48	536	91
5.7	293	311	0.14	298	2.92	303	4.03	294	3.55	291.74	763	147
5.8	288	323	0.14	305	2.89	303	4.01	288	4.06	286.88	839	139
5.9	279	297	0.13	286	2.77	293	3.72	279	3.09	278.98	777	83
5.10	265	281	0.14	276	2.90	315	4.01	265	1.93	264.97	453	68
6.1	138	149	0.12	142	1.28	148	1.81	141	3.30	133.09	872	158
6.2	146	162	0.11	158	1.30	157	1.83	146	2.84	140.36	779	130
6.3	145	160	0.11	149	1.19	147	1.71	145	2.94	139.85	883	119
6.4	131	138	0.12	132	1.32	132	1.88	131	2.02	128.89	714	97
6.5	161	189	0.12	180	1.30	178	1.83	162	2.61	153.01	631	159

Table 3. Computational results—problem sets A, B, C, D.

Problem number	Balas and Ho (BH)			Vasko and Wilson			Lagrangian heuristic				
	Optimal solution value	SCHEURI		SCFUNCITO7		Solution value (Z_{UB})	Maximum lower bound (Z_{max})	Number of subgradient iterations	Number of columns		
		Solution value	Time	Solution value	Time						
A.1	253	267	0.24	260	5.95	278	8.27	255	246.63	753	373
A.2	252	281	0.26	263	6.07	282	8.25	256	247.25	826	380
A.3	232	244	0.26	244	5.61	248	7.85	234	227.80	601	331
A.4	234	257	0.24	241	5.70	252	7.81	235	231.12	859	244
A.5	236	245	0.25	239	5.90	242	8.20	237	234.85	636	205
B.1	69	82	0.46	72	4.40	78	6.20	70	64.38	811	251
B.2	76	91	0.47	80	4.61	84	6.27	77	69.21	850	332
B.3	80	87	0.46	82	4.56	85	6.30	80	74.08	720	264
B.4	79	82	0.48	81	4.67	85	6.41	80	71.11	780	363
B.5	72	78	0.45	72	4.53	82	6.28	72	67.60	845	212
C.1	227	242	0.44	235	9.92	263	13.49	230	223.70	835	401
C.2	219	240	0.42	224	10.09	232	13.56	223	212.71	1021	577
C.3	243	266	0.41	256	9.23	259	12.74	251	234.42	968	788
C.4	219	247	0.40	227	9.62	227	13.32	224	213.63	929	574
C.5	215	228	0.40	219	9.75	228	13.46	217	211.44	798	378
D.1	60	63	0.75	62	7.11	65	9.59	61	55.22	858	345
D.2	66	72	0.74	68	6.91	68	9.96	68	59.15	866	449
D.3	72	78	0.76	74	7.54	80	10.17	75	64.97	952	495
D.4	62	65	0.79	64	7.33	70	10.12	64	55.66	649	437
D.5	61	65	0.80	63	7.27	63	10.29	62	58.56	935	227

Table 5. Computational results—problem set VW.

Problem number	Balas and Ho (BH)			SCHEURI		SCFUNC1T07		Lagrangian heuristic				
	Optimal solution value	Solution		Solution value	Time	Solution value	Time	Solution value (Z_{UB})	Time	Maximum		
		value	Time							lower bound (Z_{max})	Number of subgradient iterations	Number of columns
VW.1	188	200	0.03	193	0.27	194	0.40	188	0.74	186.98	608	41
VW.2	96	99	0.05	99	0.29	96	0.41	96	0.90	90.92	640	56
VW.3	46	47	0.06	46	0.23	46	0.34	46	0.81	42.36	668	38
VW.4	29	29	0.08	30	0.22	29	0.30	29	0.70	26.66	613	29
VW.5	154	171	0.05	166	0.49	163	0.68	154	1.22	151.58	696	60
VW.6	53	61	0.08	53	0.42	53	0.62	53	0.56	53.00	485	23
VW.7	35	39	0.11	35	0.36	37	0.52	35	1.12	33.41	861	38
VW.8	21	22	0.11	22	0.32	22	0.44	21	0.65	20.07	616	24
VW.9	107	108	0.07	108	0.67	118	0.92	107	1.17	106.24	658	58
VW.10	43	44	0.13	45	0.61	44	0.94	44	0.93	42.17	598	53
VW.11	21	22	0.14	22	0.49	21	0.71	21	1.02	18.64	731	49
VW.12	17	18	0.21	17	0.49	17	0.65	17	0.97	14.30	596	49
VW.13	165	173	0.10	166	0.92	190	1.24	165	0.84	165.00	387	31
VW.14	49	49	0.17	49	0.76	49	1.12	49	1.89	45.80	928	64
VW.15	27	27	0.19	27	0.73	27	1.03	27	2.03	24.66	995	56
VW.16	19	19	0.24	19	0.61	19	0.88	19	1.76	16.30	717	57
VW.17	365	397	0.05	371	0.56	411	0.74	366	2.29	346.19	669	152
VW.18	123	143	0.07	126	0.43	126	0.62	126	2.73	113.63	957	92
VW.19	69	71	0.11	69	0.39	69	0.57	69	1.68	64.79	726	54
VW.20	26	26	0.12	26	0.34	26	0.47	26	1.15	25.33	781	17
VW.21	204	234	0.08	232	0.85	216	1.24	204	2.93	202.32	1022	76
VW.22	86	96	0.13	93	0.75	93	1.06	87	3.25	75.91	869	132
VW.23	47	51	0.18	52	0.64	47	0.93	47	2.76	40.21	889	84
VW.24	30	31	0.22	30	0.55	30	0.78	30	3.00	26.59	1018	59
VW.25	170	176	0.13	176	1.33	175	1.87	171	2.29	164.22	699	126
VW.26	50	56	0.19	54	0.96	54	1.39	50	2.29	46.18	879	74
VW.27	—	—	—	—	—	—	—	—	—	—	—	—
VW.28	21	22	0.31	21	0.78	21	1.13	21	2.32	18.64	854	51
VW.29	50	51	0.13	50	1.22	55	1.64	50	1.02	49.66	538	48
VW.30	20	22	0.21	22	1.03	22	1.48	21	1.51	18.68	801	71
VW.31	15	16	0.33	16	0.99	16	1.40	15	1.37	13.59	635	57
VW.32	9	10	0.32	10	0.84	10	1.20	9	1.50	7.35	827	50

Note: (a) the details of problem number VW.27 have been lost (see [20]); (b) discrepancies between the solution values for SCHEURI and SCFUNC1T07 given above and those given in [20] can be accounted for by:

- (1) the modification made to SCHEURI and SCFUNC1T07 (see text),
- (2) different random numbers (SCFUNC1T07),
- (3) the fact that the same code will give different results depending upon how the rows/columns are indexed (SCHEURI, SCFUNC1T07 and the Lagrangian heuristic).

use 4.1 to refer to the first problem in problem set 4, 4.2 to refer to the second problem, etc. Problem sets 4–6, A–D, and VW are ones for which the value of the optimal solution is known (from [4, 18, 20]) whilst problem sets E–H are designed to investigate the behavior of the heuristic algorithms on: (a) relatively dense problems (problem sets E and F); and (b) large sparse problems (problem sets G and H).

Problems of the size corresponding to problem sets F and H are effectively the largest problems our FORTRAN code could solve within the 1.8MWords of user memory available on the Cray-1S.

In Tables 2–5 we give, for each problem:

- (a) the value of the optimal solution (if known)
- (b) for the Balas and Ho [3] and Vasko and Wilson [17] heuristics the value of the best feasible solution found and the associated computation time (in Cray-1S seconds)
- (c) for the Lagrangian heuristic presented in this article the value of the best feasible solution found, the associated computation time (in Cray-1S seconds), the value of the maximum lower bound found, the number of subgradient iterations, and the number of columns left at the end of the heuristic (excluding those eliminated from the problem).

As the Cray-1S is a vector processing computer we, in order to estimate the effect of this vector processing capability (for our FORTRAN code), inhibited that capability and resolved problem G.3 (the most time-consuming problem overall). This increased the computation time by a factor of 1.10 for BH, by factors of 1.37 and 1.35 for SCHEURI and SCFUNC1TO7, and by a factor of 1.24 for the Lagrangian heuristic.

Examining Tables 2–5 we see that:

- (1) SCHEURI gave a better (or equal) solution to SCFUNC1TO7 for all but 19 of the 96 test problems.
- (2) SCHEURI gave a better (or equal) solution to BH for all but six of the 96 test problems, albeit at a greater computational cost. This tradeoff between computational cost and quality of solution is common in heuristic algorithms.
- (3) The Lagrangian heuristic presented in this article gave a better (or equal) solution to BH or SCHEURI or SCFUNC1TO7 for all but three of the 96 test problems. Over all 96 test problems the total computation time for BH was 72.08 Cray-1S seconds, for SCHEURI it was 668.35 Cray-1S seconds, for SCFUNC1TO7 it was 923.77 Cray-1S seconds and for the Lagrangian heuristic it was 1064.41 Cray-1S seconds.
- (4) For the 76 test problems for which the optimal solution is known BH found the optimal solution five times, SCHEURI found the optimal solution 13 times, SCFUNC1TO7 found the optimal solution 14 times and the Lagrangian heuristic found the optimal solution 44 times. The average percentage deviation from optimal $[100(\text{solution value} - \text{optimal})/\text{optimal}]$ was 7.416% for BH, 6.428% for SCFUNC1TO7, 3.311% for SCHEURI, and 0.638% for the Lagrangian heuristic.

Comparing Table 2 with the corresponding tables in Balas and Ho [3] we have that the Lagrangian heuristic gave a better (or equal) solution to the best solution of *any* of the heuristics used in that article (before first branching—see [3]) for all but one of the 25 test problems in problem sets 4–6.

Comparing Table 5 with the corresponding table in Vasko and Wolf [20] we have that the Lagrangian heuristic gave a better (or equal) solution to the best solution of *any* of the heuristics used in that article for all but two of the 31 test problems in problem set VW.

5. CONCLUSIONS

In this article we have presented a heuristic for the set-covering problem based upon Lagrangian relaxation and subgradient optimization. Computational results indicated that this heuristic gave better-quality results than a number of other heuristics, albeit at greater computational cost.

ACKNOWLEDGMENT

The author would like to acknowledge the help of K. L. Stott and F. E. Wolf (Bethlehem Steel Corporation) and F. J. Vasko (Kutztown University) who supplied some of the test problems used in this article.

REFERENCES

- [1] Baker, E. K., "Efficient Heuristic Algorithms for the Weighted Set Covering Problem," *Computers and Operations Research*, 8(4), 303-310 (1981).
- [2] Baker, E., and Fisher, M., "Computational Results for Very Large Air Crew Scheduling Problems," *Omega*, 9(6), 613-618 (1981).
- [3] Balas, E., and Ho, A., "Set Covering Algorithms Using Cutting Planes, Heuristics, and Subgradient Optimization: A Computational Study," *Mathematical Programming*, 12, 37-60 (1980).
- [4] Beasley, J. E., "An Algorithm for Set Covering Problems," *European Journal of Operational Research*, 31, 85-93 (1987).
- [5] Beasley, J. E., "Lagrangian Heuristics for Location Problems," Working Paper, School of Management, Imperial College, London SW7 2AZ, England, 1988.
- [6] Christofides, N., and Korman, S., "A Computational Survey of Methods for the Set Covering Problem," *Management Science*, 21(5), 591-599 (1975).
- [7] Chvatal, V., "A Greedy Heuristic for the Set-Covering Problem," *Mathematics of Operations Research*, 4(3), 233-235 (1979).
- [8] Etcheberry, J., "The Set-Covering Problem: A New Implicit Enumeration Algorithm," *Operations Research*, 25(5), 760-772 (1977).
- [9] Fisher, M. L., "The Lagrangian Relaxation Method for Solving Integer Programming Problems," *Management Science*, 27(1), 1-18 (1981).
- [10] Fisher, M. L., "An Applications Oriented Guide to Lagrangian Relaxation," *Interfaces*, 15(2), 10-21 (1985).
- [11] Ho, A. C., "Worst Case Analysis of a Class of Set Covering Heuristics," *Mathematical Programming*, 23, 170-180 (1982).
- [12] Hochbaum, D. S., "Approximation algorithms for the weighted set covering and node cover problems," Working Paper No. 64-79-80, Graduate School of Industrial Administration, Carnegie-Mellon University, Pittsburgh, PA, 1980.
- [13] Hochbaum, D. S., "Approximation Algorithms for the Set Covering and Vertex Cover Problems," *SIAM Journal of Computing*, 11(3), 555-556 (1982).
- [14] Karp, R. M., "Reducibility among Combinatorial Problems," in R. E. Miller and J. W. Thatcher, Eds., *Complexity of Computer Computations*, Plenum Press, New York, 1972.
- [15] Paixao, J., "Algorithms for Large Scale Set Covering Problems," Ph.D. thesis, School of Management, Imperial College, London SW7 2AZ, England, 1984.
- [16] Roth, R., "Computer Solutions to Minimum-Cover Problems," *Operations Research*, 17(3), 455-465 (1968).
- [17] Vasko, F. J., and Wilson, G. R., "An Efficient Heuristic for Large Set Covering Problems," *Naval Research Logistics Quarterly*, 31, 163-171 (1984).
- [18] Vasko, F. J., and Wilson, G. R., "Using a Facility Location Algorithm to Solve Large Set Covering Problems," *Operations Research Letters*, 3(2), 85-90 (1984).

- [19] Vasko, F. J., and Wilson, G. R., "Hybrid Heuristics for Minimum Cardinality Set Covering Problems," *Naval Research Logistics Quarterly*, **33**, 241–249 (1986).
- [20] Vasko, F. J., and Wolf, F. E., "Solving Large Set Covering Problems on a Personal Computer," *Computers and Operations Research*, **15**(2), 115–121 (1988).

Manuscript received June 17, 1988

Revised manuscript received July 13, 1988

Accepted April 3, 1989